- The fewer details you need to worry about at once, the easier it is to understand code. It's easier to focus on the logic for allowing users access to a specific project if it's not thrown together in the same file with code for finding users, code to display full names, and code to list a user's undone tasks.

It's possible to overdo isolation to the point where the pieces of code are so small that your cognitive load goes back up as you try to put the pieces together. You probably aren't near that boundary yet.

Isolating in a Rails application like this is often mischaracterized as setting up your application to switch away from Rails. That's not the point. If you do need to switch from Rails, you're probably looking at significant code changes no matter how you organize your code. The point is to make your application best able to handle both the complexity in the current application and the inevitable changes over time.

That said, Rails itself does change. And it's even been known to throw in changes that are not backward-compatible. When that happens, the less intertwined your application logic is with Rails logic, the easier your upgrade path.

However, the odds that this approach will cause the founder of Rails to call you an architecture astronaut remain pretty high.

Prescription 49	This is your regular reminder that software is complex and there are multiple paths to success.

Rails Test Prescriptions, Hold the Rails

The basic idea is isolation: isolating your objects from each other and isolating your code from Rails functionality that would require Rails to be loaded. Isolation means that different objects interact with each other over as small a set of methods as possible. It means objects, ideally, know nothing about the internal structure of other objects in the system.

This doesn't have to be all that complicated. You've been writing tests with reasonably good habits so far. To isolate your `Project` tests from anything other than ActiveRecord, all you need to do is change the header. Remove the `require rails_helper` call and replace it:

environment/01/test/models/project_test.rb

```
require_relative "../active_record_test_helper"
require_relative "../../app/models/project"
require_relative "../../app/models/task"
```

Where once you had a single `require`, now you have three.